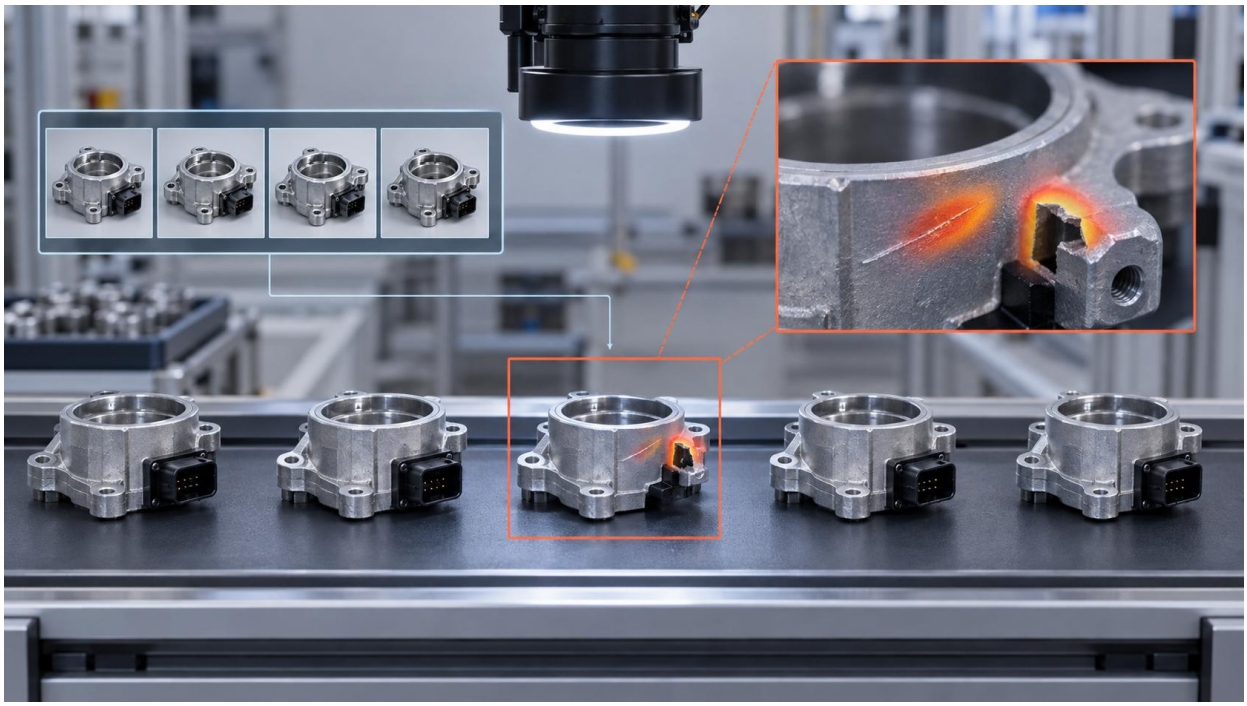
Project 2: Few-Shot Industrial Visual Anomaly Detection

AIAA 3201—Introduction to Computer Vision

Term Project Instruction

Spring 2027



1 Overview

Core Task: Given a small set of defect-free reference images, build a system that detects anomalous product images and localizes defects. Complete one simple baseline, reproduce AnomalyDINO, and implement one improvement. AnomalyCLIP and additional robustness tests are optional extensions.

Key Concepts:

- **Unsupervised Anomaly Detection:** Learning normality without labeled defects.
- **Foundation Model Features:** Patch-level representation and nearest-neighbor retrieval.
- **Vision-Language Models:** Normal/abnormal prompt learning and zero-shot transfer.
- **Reliability:** Domain shift, calibration, uncertainty, and false-alarm control.

Group Size: 2 students per group.

2 Submission Requirements

- **PDF Report (Mandatory):**
  - **Format:** Use the CVPR LaTeX Template. The length must be 6–8 pages (excluding references).CVPR template
  - **ArXiv Upload (Optional):** Upload your final report to arXiv. Remember to change the "REVIEW version" to "CAMERA-READY version" in the template. Please clearly indicate your arXiv ID on the first page of the Canvas submission.

- **Content Requirements:**
  1. **Abstract & Introduction:** Background, motivation, problem definition, and solution overview. Explicitly include your public GitHub repository link at the end of the abstract.
  2. **Related Work:** Review and cite at least all papers listed in the Recommended Reading List (Section 4). Expanding beyond this list is encouraged.
  3. **Method:** Explain the technical roadmap in your own words. Use high-quality pipeline diagrams and visual examples to support the explanation.
  4. **Experiments:** Present quantitative results in tables, qualitative comparisons in figures, ablation studies, and categorized failure cases.
  5. **Conclusion:** Summarize the findings, limitations, and specific directions for future work.
- **Code (Mandatory):** Upload the code to a public GitHub repository with a clear README.md. Include dependencies, data preparation, model weights, training and evaluation commands, and representative visual results. Do not submit raw code files to Canvas.
- **Interactive Demo (Mandatory):** Submit a runnable demo or a recorded demonstration that accepts a product image and displays an image-level anomaly score, an OK/NG decision, and a pixel-level heatmap. Pack the demo video and representative outputs into anomaly_demo.zip.

Important Tips for Success

- **Method Flexibility:** You may follow the suggested roadmap or design another technically sound method. Strict adherence to the Implementation Roadmap is not required if the core task and evaluation protocol are fulfilled.
- **Completeness:** Complete Parts 1 and 2, then choose ONE improvement in Part 3. Compare it with the unchanged Part 2 method. If it does not improve the main metric, explain why using controlled experiments and failure cases. Ignoring Part 3 will result in an "Incomplete" grade.
- **Visuals:** Clean pipeline diagrams, prediction overlays, zoomed comparisons, and failure visualizations are essential in a Computer Vision course and will substantially affect the presentation score.
- **Dataset Scope:** Use only the six VisA categories in Section 6.1. Evaluating all 12 is optional. Assessment emphasizes the reasoning and originality of the improvement, measured results, and experimental analysis; a larger dataset is not required for a high score.
- **Fair Comparison:** Use identical data splits, preprocessing, input resolution, and evaluation code when comparing methods. Report runtime, peak GPU memory, and all test-time refinements.
- **Citations:** Cite all datasets, pretrained models, external repositories, and borrowed implementation components correctly.

3 Guide to Start

To help you get started efficiently, we recommend the following workflow:

1. **Step 1: Literature Review.** Read the papers in the Recommended Reading List (Section 4). Understanding normal-only learning, patch embeddings, nearest-neighbor anomaly scoring, and image-level versus pixel-level metrics is essential before coding.
2. **Step 2:** Code Familiarization. Start with AnomalyDINO and its default DINOv2 ViT-S/14 backbone. Run one category first. AnomalyCLIP is optional. | AnomalyDINO
3. **Step 3: Experimentation.** Download the datasets from the official links in Section 6.1. Run the baselines and official pretrained methods using the stated category subset, data split, and random seeds. Cache frozen backbone features and save anomaly maps and error cases for every method.
4. **Step 4: Writing & Submission.** Organize quantitative results, ablations, and failure cases into tables and figures. Write the report with the CVPR template, cite all references, and submit the required files to Canvas.

4 Recommended Reading List

- **Foundations:** MVTec AD [1], PaDiM [2], PatchCore [3]
- **Reconstruction & Synthesis:** DRAEM [4], CutPaste [5]
- **Foundation Models:** DINOv2 [6], AnomalyCLIP [7], AnomalyDINO [8]
- **Efficiency & Robustness:** EfficientAD [9], MVTec AD 2 [10]

5 Implementation Roadmap

5.1 Part 1: The Baseline – "Classical Normality Modeling"

Goal: Build a complete normal-only inspection pipeline and understand how alignment, representation, and threshold selection affect anomaly detection.

1. Required Baseline: Template Difference

- **Step 1:** Reference Images. Use the same normal reference images as AnomalyDINO. Resize to the same resolution and compare each test image with the closest template. Alignment with ECC or feature matching is optional.
- **Step 2: Difference Map.** Compute absolute RGB/gray differences, SSIM differences, gradients, or local texture descriptors. Use smoothing and morphology to obtain anomaly regions.
- **Step 3:** Demo Threshold. For each category, reserve the first 20 normal training files in sorted order for calibration; exclude them from every reference selection. Use the 95th percentile of their image scores as the OK/NG threshold. These extra normal images are for demo calibration only and do not enter the memory bank or fit the model.

2. Optional Extension: Convolutional Autoencoder

- **Algorithm:** If desired, train a compact autoencoder on normal training images and use reconstruction error as an anomaly map. State its training-image count separately; this is not a mandatory few-shot comparison.
- **Key Analysis:** Explain where the required template baseline confuses changes in pose, background, or illumination with defects.

Expected Result: The baseline should detect obvious scratches and missing components but will often confuse pose, background, and illumination changes with defects.

5.2 Part 2: SOTA Reproduction – "Foundation Features & Prompt Learning"

Goal: Reproduce a recent, openly released method that works with very few normal reference images and produces both image-level and pixel-level predictions.

1. Few-Shot Anomaly Detection: AnomalyDINO [8]

- **System:** Extract DINOv2 patch features from 1/2/4/8 normal references. Construct a memory bank and score each test patch by its nearest normal neighbor.
- **Required Reproduction:** Use the official code with dinov2_vits14, resolution 448, and agnostic preprocessing. Evaluate 1/2/4/8 normal references with seeds 0, 1, and 2. Save the reference filenames and reuse them for your baseline and improvement. Exclude the calibration images before sampling.
- **Key Learning Point:** Analyze why frozen semantic features can localize unseen defects without task-specific training, and where coarse patch resolution fails.

2. Optional Extension: AnomalyCLIP [7]

- **System:** AnomalyCLIP offers a text-guided comparison. If selected, use the repository checkpoint intended for evaluation on VisA and disclose its source training data; do not train or select prompts using VisA test labels.
- **Comparison:** This extension is optional. The required comparison is template difference, official AnomalyDINO, and your improved AnomalyDINO under the same data protocol.

Expected Result: AnomalyDINO should provide a strong training-free few-shot baseline; AnomalyCLIP may generalize across categories but can be sensitive to prompts and fine-grained defects.

5.3 Part 3: Exploration – "Robustness & Reliable Decisions"

Goal: Identify one concrete failure mode and implement ONE justified improvement. The directions below are alternatives, not a checklist. Keep the backbone frozen unless you explicitly study a small adapter.

- **Direction A (Multi-Scale Features):** Fuse patch features from multiple DINOv2 layers or image scales. Test whether fine features improve tiny scratches without increasing background false positives.
- **Direction B (Background Suppression):** Use foreground masks, PCA masks, or segmentation prompts to remove unstable background patches from the memory bank and anomaly map.
- **Direction C (Synthetic Defects):** Generate CutPaste or texture-based pseudo-defects and train only a lightweight scoring head. Study which synthetic anomalies transfer to real defects.
- **Direction D (Prompt & Feature Fusion):** Combine AnomalyCLIP prompt scores with AnomalyDINO visual distances using a calibrated gate or uncertainty-aware weighting.
- **Direction E (Memory & Efficiency):** Compress the normal memory bank with clustering, coreset selection, or product-specific prototypes. Report accuracy, runtime, and memory trade-offs.
- **Direction F (Selective Prediction):** Calibrate anomaly scores and allow an "uncertain" output. Evaluate risk-coverage curves and the false-negative rate at a fixed false-positive rate.

6 Dataset & Evaluation

6.1 Datasets and Required Downloads

1. **VisA (Only Required Dataset):** Download the public archive, approximately 1.9 GB. No application form is required. Use candle, capsules, cashew, chewinggum, fryum, and pipe_fryum. Keep the official 1cls normal-training/test split; all test images and their masks are used for evaluation [11]. | Dataset archive | Official data preparation
2. **Preparation and Starter Code:** Extract to VisA. In the official spot-diff repository, run: python utils/prepare_data.py --split-type 1cls --data-folder ./VisA --save-folder ./VisA_pytorch --split-file ./split_csv/1cls.csv. Set AnomalyDINO --data_root to VisA_pytorch/1cls/. In src/utils.py, restrict the VisA object list returned by get_dataset_info to the six categories above. Use --dataset VisA --shots 1 2 4 8 --num_seeds 3 --preprocess agnostic --eval_segm --faiss_on_cpu. DINOv2 weights download through the code. A CUDA GPU is recommended; --faiss_on_cpu moves only neighbor search to CPU. | AnomalyDINO setup and commands | DINOv2 weights | Optional AnomalyCLIP

6.2 Metrics (Mandatory)

- **Image-Level Detection:** Image AUROC is the main metric. Report each category and the unweighted mean across the six categories. Use image scores before thresholding.
- **Pixel-Level Localization:** Report pixel AUROC using all test images, including normal images with all-zero masks. Compute within each category, then average categories equally. AP and AUPRO are optional; if reporting AUPRO, use the official 0.30 false-positive-rate limit.
- **Few-Shot Stability:** For each shot count, report the mean and standard deviation across seeds 0, 1, and 2. Compare the official and improved method using identical reference selections.
- **Efficiency:** Report average inference time per image and peak GPU memory on the same device; identify the input resolution and whether feature extraction is included.
- **Qualitative Evaluation:** Show input, mask, and anomaly heatmap for five representative difficult examples. Explain actual errors or remaining limitations; do not manufacture a fixed number of failures.

Note: Use only training/calibration images to choose settings. Test labels are for final evaluation. Do not rescale scores separately for each test image when computing AUROC.

References

[1] Bergmann, P., et al. “MVTec AD—A Comprehensive Real-World Dataset for Unsupervised Anomaly Detection.” CVPR 2019.

[2] Defard, T., et al. “PaDiM: A Patch Distribution Modeling Framework for Anomaly Detection and Localization.” ICPR Workshops 2021.

[3] Roth, K., et al. “Towards Total Recall in Industrial Anomaly Detection.” CVPR 2022.

[4] Zavrtanik, V., et al. “DRAEM—A Discriminatively Trained Reconstruction Embedding for Surface Anomaly Detection.” ICCV 2021.

[5] Li, C.-L., et al. “CutPaste: Self-Supervised Learning for Anomaly Detection and Localization.” CVPR 2021.

[6] Oquab, M., et al. “DINOv2: Learning Robust Visual Features without Supervision.” TMLR 2024.

[7] Zhou, Q., et al. “AnomalyCLIP: Object-Agnostic Prompt Learning for Zero-Shot Anomaly Detection.” ICLR 2024.

[8] Damm, S., et al. “AnomalyDINO: Boosting Patch-Based Few-Shot Anomaly Detection with DINOv2.” WACV 2025.

[9] Batzner, K., et al. “EfficientAD: Accurate Visual Anomaly Detection at Millisecond-Level Latencies.” WACV 2024.

[10] Heckler-Kram, L., et al. “The MVTec AD 2 Dataset: Advanced Scenarios for Unsupervised Anomaly Detection.” IJCV 2026.

[11] Zou, Y., et al. “SPot-the-Difference Self-Supervised Pre-Training for Anomaly Detection and Segmentation.” ECCV 2022.

[12] Kirillov, A., et al. “Segment Anything.” ICCV 2023.